



CS 230 - Distributed Systems

Lecture 1 - Introduction to Distributed Systems
Tuesdays, Thursdays 3:30-4:50p.m.

Prof. Nalini Venkatasubramanian
nalini@ics.uci.edu

Course logistics and details

- Course Web page
 - <http://www.ics.uci.edu/~cs230>
- Lectures - TuTh 3:30-4:50p.m
- Must Read: Course Reading List
 - **Collection of Technical papers and reports by topic**
- Reference Books
 - **Distributed Systems: Concepts & Design**, 4th ed. by Coulouris et al. ISBN: 0-321-26354-5. (preferred)
 - **Distributed Systems: Principles and Paradigms**, 2nd ed. by Tanenbaum & van Steen. ISBN: 0-132-39227-5.
 - **Distributed Computing: Principles, Algorithms, and Systems**, 1st ed. by Kshemkalyani & Singhal. ISBN: 0-521-87634-6

Prerequisite Knowledge

- Necessary – Operating Systems Concepts and Principles, basic computer system architecture
- Highly Desirable – Understanding of Computer Networks, Network Protocols
- Necessary – Basic programming skills in Java, C++,...

Course logistics and details

- Homeworks
 - | Paper summaries
- Midterm Examination
- Course Project
 - | Maybe done individually or in groups
 - | **Project proposal due end of Week 2**
 - | **Survey of related research due end of Week 6**
 - | **Final Project presentations/demos/reports – Finals week**
 - | Potential projects will be available on webpage

CompSci 230 Grading Policy

- Homeworks - 30% of final grade
 - | **1 paper summary due every week after Week 2 covering topics discussed the previous week.**
- Midterm - 30% of final grade
 - | **Tentatively in Week 7**
- Class Project - 40% of the final grade
- Final assignment of grades will be based on a curve.

Lecture Schedule



- | Weeks 1,2,3: Distributed Systems Fundamentals
 - | **Introduction – Needs/Paradigms**
 - Basic Concepts and Terminology, Concurrency
 - | **Time and State in Distributed Systems**
 - Physical and Logical Clocks
 - Distributed Snapshots, Termination Detection, Consensus
- | Week 4,5,6: Distributed OS and Middleware Issues
 - | **Interprocess Communication**
 - Remote Procedure Calls, Distributed Shared Memory
 - | **Distributed Process Coordination/Synchronization**
 - Distributed Mutual Exclusion/Deadlocks, Leader Election
 - | **Distributed Process and Resource Management**
 - Task Migration, Load Balancing
 - | **Distributed I/O and Storage Subsystems**
 - Distributed FileSystems

Lecture Schedule

- Weeks 7,8: Messaging and Communication in Distributed Systems
 - | **Naming in Distributed Systems**
 - | **Gossip, Tree, Mesh Protocols**
 - | **Group Communication**
- Weeks 9,10: Non-functional “ilities” in distributed systems
 - | **Reliability and Fault Tolerance**
 - | **Quality of Service and Real-time Needs**
- Sample Distributed Systems (time permitting)
 - | **P2P, Grid and Cloud Computing, Mobile/Pervasive**

What is not covered



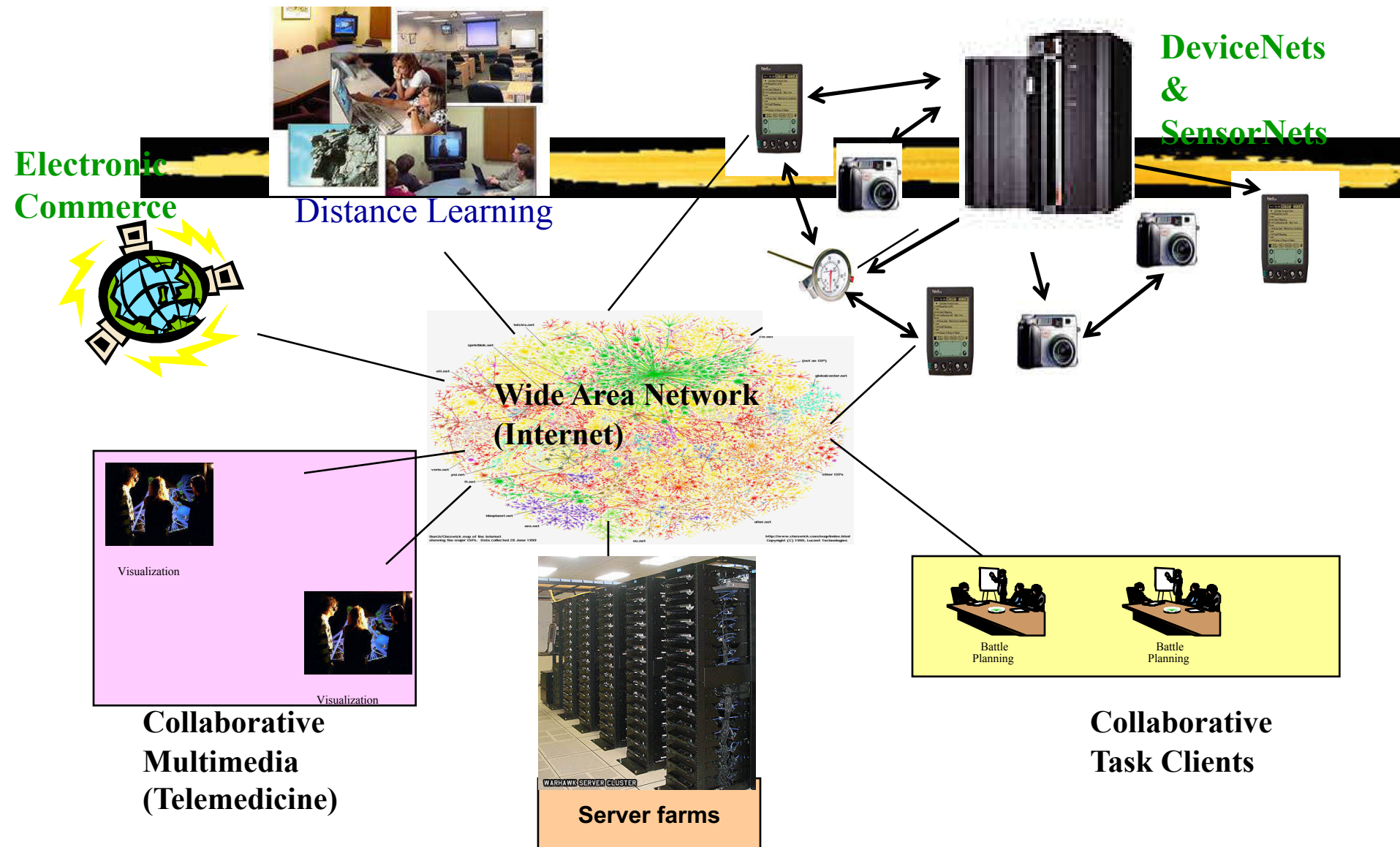
- Security in Distributed Systems (Prof. Tsudik's course)
- Distributed Database Management and Transaction Processing (CS 223)
- Distributed Objects and Middleware Platforms (CS237)

Introduction

■ Distributed Systems

- Multiple independent computers that appear as one
- Lamport's Definition
 - | **" You know you have one when the crash of a computer you have never heard of stops you from getting any work done."**
- "A number of interconnected autonomous computers that provide services to meet the information processing needs of modern enterprises."

Next Generation Information Infrastructure



Requirements - Availability, Reliability, Quality-of-Service, Cost-effectiveness, Security

Characterizing Distributed Systems

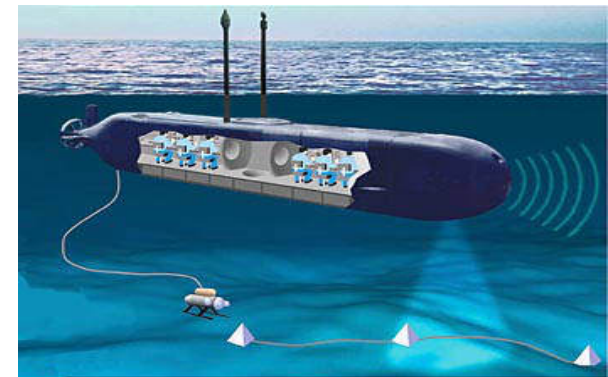
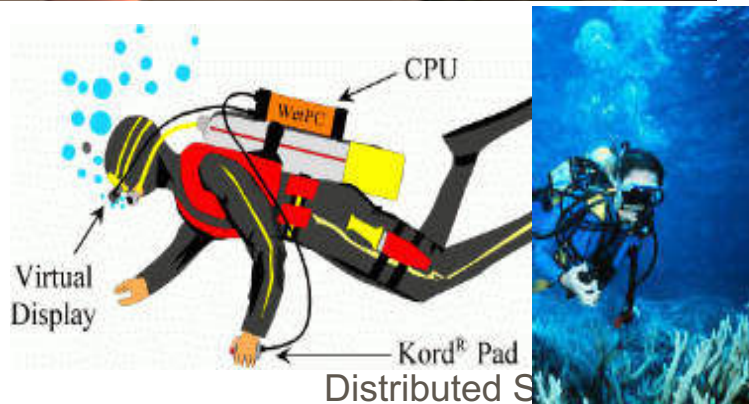
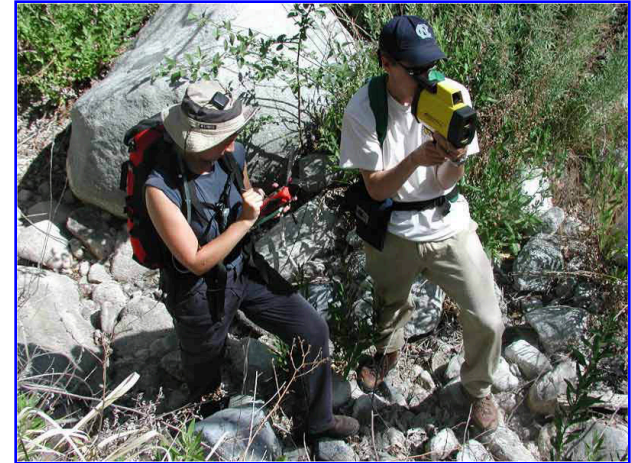
- Multiple Autonomous Computers
 - each consisting of CPU's, local memory, stable storage, I/O paths connecting to the environment
 - Geographically Distributed
- Interconnections
 - some I/O paths interconnect computers that talk to each other
- Shared State
 - No shared memory
 - systems cooperate to maintain shared state
 - maintaining global invariants requires correct and coordinated operation of multiple computers.

Examples of Distributed Systems

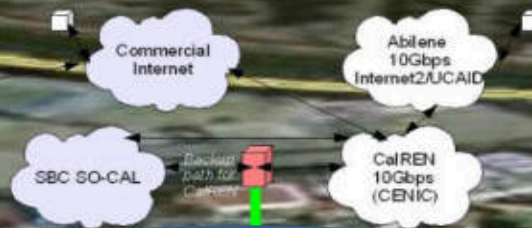
- Transactional applications - Banking systems
- Manufacturing and process control
- Inventory systems
- General purpose (university, office automation)
- Communication – email, IM, VoIP, social networks
- Distributed information systems
 - WWW
 - Cloud Computing Infrastructures
 - Federated and Distributed Databases



Mobile & ubiquitous distributed systems



A Distributed CyberPhysical Space – UCI Responsphere



*Campus-wide infrastructure to instrument, experiments,
monitor, disaster drills & to validate technologies*

sensing, communicating, storage & computing infrastructure

Software for real-time collection, analysis, and processing of
sensor information

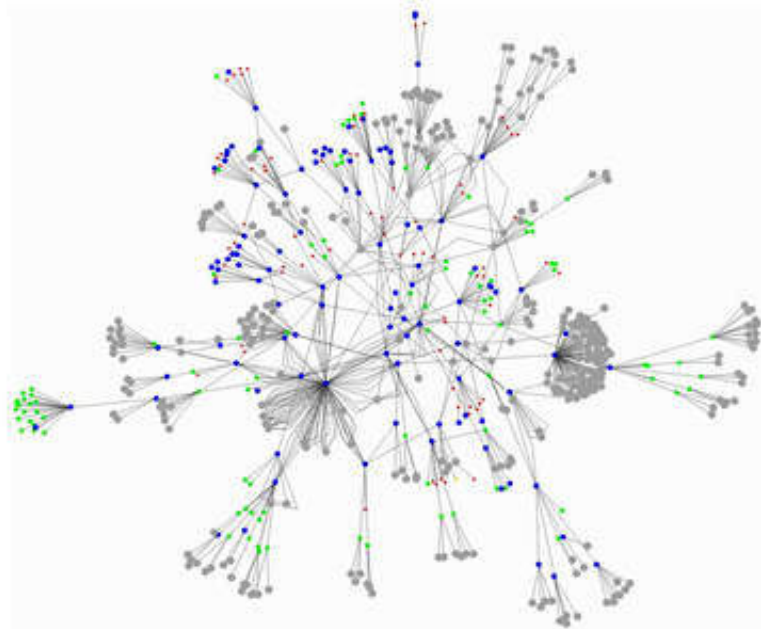
used to create real time information awareness & post-drill
analysis

People
counters

Mobile
cameras

responsphere

Peer to Peer Systems



P2P File Sharing

Napster, Gnutella, Kazaa, eDonkey,
BitTorrent
Chord, CAN, Pastry/Tapestry,
Kademlia

P2P Communications

MSN, Skype, Social Networking Apps

P2P Distributed Computing

Seti@home

Use the **vast resources** of machines at **the edge of the Internet** to build a network that allows resource sharing **without any central authority**.

Why Distributed Computing?

- Inherent distribution
 - Bridge customers, suppliers, and companies at different sites.
- Speedup - improved performance
- Fault tolerance
- Resource Sharing
 - Exploitation of special hardware
- Scalability
- Flexibility

Why are Distributed Systems Hard?

- Scale
 - numeric, geographic, administrative
- Loss of control over parts of the system
- Unreliability of message passing
 - unreliable communication, insecure communication, costly communication
- Failure
 - Parts of the system are down or inaccessible
 - Independent failure is desirable

Design goals of a distributed system

- Sharing
 - HW, SW, services, applications
- Openness(extensibility)
 - use of standard interfaces, advertise services, microkernels
- Concurrency
 - compete vs. cooperate
- Scalability
 - avoids centralization
- Fault tolerance/availability
- Transparency
 - location, migration, replication, failure, concurrency

Classifying Distributed Systems

- Based on degree of synchrony
 - Synchronous
 - Asynchronous
- Based on communication medium
 - Message Passing
 - Shared Memory
- Fault model
 - Crash failures
 - Byzantine failures

Computation in distributed systems

- Asynchronous system
 - no assumptions about process execution speeds and message delivery delays
- Synchronous system
 - make assumptions about relative speeds of processes and delays associated with communication channels
 - constrains implementation of processes and communication
- Models of concurrency
 - Communicating processes
 - Functions, Logical clauses
 - Passive Objects
 - Active objects, Agents

Concurrency issues

- Consider the requirements of transaction based systems
 - Atomicity - either all effects take place or none
 - Consistency - correctness of data
 - Isolated - as if there were one serial database
 - Durable - effects are not lost
- General correctness of distributed computation
 - Safety
 - Liveness

Communication in Distributed Systems

- Provide support for entities to communicate among themselves
 - Centralized (traditional) OS's - local communication support
 - Distributed systems - communication across machine boundaries (WAN, LAN).
- 2 paradigms
 - Message Passing
 - **Processes communicate by sharing messages**
 - Distributed Shared Memory (DSM)
 - **Communication through a virtual shared memory.**

Message Passing

- Basic communication primitives
 - Send message
 - Receive message
- Modes of communication
 - Synchronous
 - | atomic action requiring the participation of the sender and receiver.
 - | Blocking send: blocks until message is transmitted out of the system send queue
 - | Blocking receive: blocks until message arrives in receive queue
 - Asynchronous
 - | Non-blocking send: sending process continues after message is sent
 - | Blocking or non-blocking receive: Blocking receive implemented by timeout or threads. Non-blocking receive proceeds while waiting for message. Message is queued (BUFFERED) upon arrival.

Reliability issues

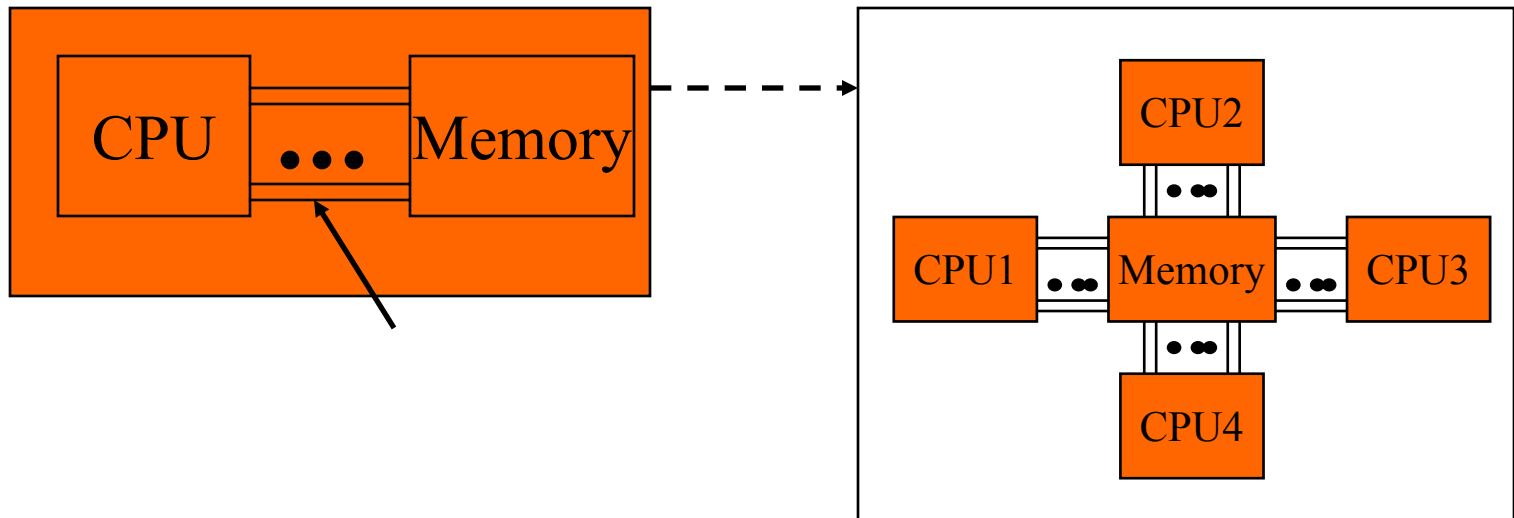
- Unreliable communication
 - Best effort, No ACK's or retransmissions
 - Application programmer designs own reliability mechanism
- Reliable communication
 - Different degrees of reliability
 - Processes have some guarantee that messages will be delivered.
 - Reliability mechanisms - ACKs, NACKs.

Remote Procedure Call

- Builds on message passing
 - extend traditional procedure call to perform transfer of control and data across network
 - Easy to use - fits well with the client/server model.
 - Helps programmer focus on the application instead of the communication protocol.
 - Server is a collection of exported procedures on some shared resource
 - Variety of RPC semantics
 - “maybe call”
 - “at least once call”
 - “at most once call”

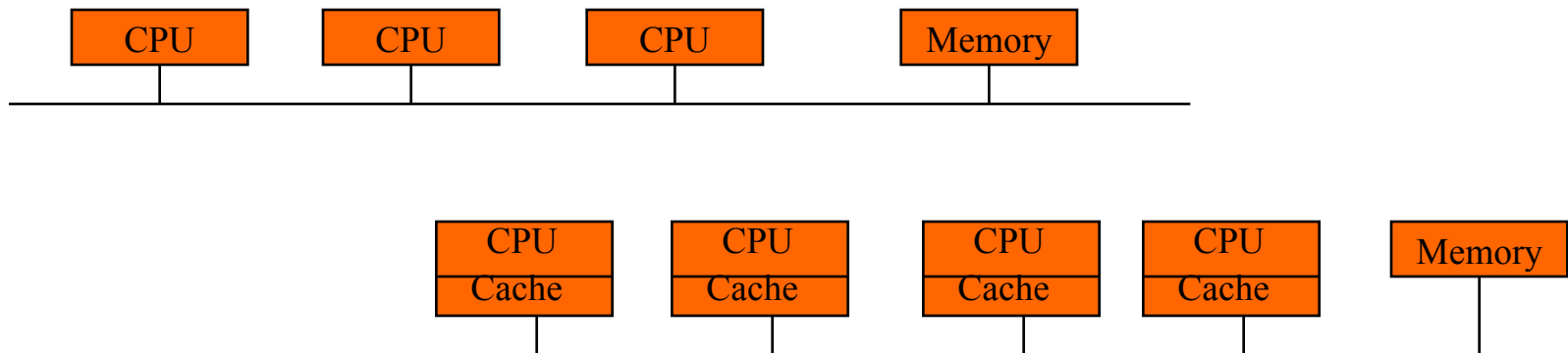
Distributed Shared Memory

- Communication Abstraction used for processes on machines that do not share memory
 - Motivated by shared memory multiprocessors that do share memory



Distributed Shared Memory

- Processes read and write from virtual shared memory.
 - Primitives - read and write
 - OS ensures that all processes see all updates
- Caching on local node for efficiency
 - Issue - cache consistency



Fault Models in Distributed Systems

■ Crash failures

- A processor experiences a crash failure when it ceases to operate at some point without any warning. Failure may not be detectable by other processors.
 - **Failstop - processor fails by halting; detectable by other processors.**

■ Byzantine failures

- completely unconstrained failures
- conservative, worst-case assumption for behavior of hardware and software
- covers the possibility of intelligent (human) intrusion.

Other Fault Models in Distributed Systems

- Dealing with message loss
 - Crash + Link
 - | Processor fails by halting. Link fails by losing messages but does not delay, duplicate or corrupt messages.
 - Receive Omission
 - | processor receives only a subset of messages sent to it.
 - Send Omission
 - | processor fails by transmitting only a subset of the messages it actually attempts to send.
 - General Omission
 - | Receive and/or send omission

Other Distributed System issues

- Concurrency and Synchronization
- Distributed Deadlocks
- Time in distributed systems
- Naming
- Replication
 - improve availability and performance
- Migration
 - of processes and data
- Security
 - eavesdropping, masquerading, message tampering, replaying

Client/Server Computing

- Client/server computing allocates application processing between the client and server processes.
- A typical application has three basic components:
 - Presentation logic
 - Application logic
 - Data management logic

Client/Server Models

- There are at least three different models for distributing these functions:
 - Presentation logic module running on the client system and the other two modules running on one or more servers.
 - Presentation logic and application logic modules running on the client system and the data management logic module running on one or more servers.
 - Presentation logic and a part of application logic module running on the client system and the other part(s) of the application logic module and data management module running on one or more servers

Distributed Computing Environment (DCE)

- DCE is from the Open Software Foundation (OSF), and now X/Open, offers an environment that spans multiple architectures, protocols, and operating systems.
 - DCE supported by major software vendors.
- It provides key distributed technologies, including RPC, a distributed naming service, time synchronization service, a distributed file system, a network security service, and a threads package.

Distributed Systems Middleware



- Middleware is the software between the application programs and the operating System and base networking
- Integration Fabric that knits together applications, devices, systems software, data
- Middleware provides a comprehensive set of higher-level distributed computing capabilities and a set of interfaces to access the capabilities of the system.

Distributed Systems Middleware

- Enables the modular interconnection of distributed software
 - | **abstract over low level mechanisms used to implement resource management services.**
- Computational Model
 - | **Support separation of concerns and reuse of services**
- Customizable, Composable Middleware Frameworks
 - | **Provide for dynamic network and system customizations, dynamic invocation/revocation/installation of services.**
 - | **Concurrent execution of multiple distributed systems policies.**

Distributed Object Computing

- Combining distributed computing with an object model.
 - Allows software reusability and a more abstract level of programming
 - The use of a broker like entity or bus that keeps track of processes, provides messaging between processes and other higher level services
 - Examples
 - **CORBA, JINI, EJB, J2EE**

The Evergrowing Middleware Alphabet Soup

Distributed
Computing
Environment (DCE)



WS-BPEL
WSIL



Adaptive



Orbix

WSDL

Java Transaction API (JTA)

JNDI

JMS

LDAP

IOP

IIOP

GIOP

BPEL

Object Request Broker
(ORB)



BEA Tuxedo®

RTCORBA

EAI

Message Queuing (MSMQ)

XQuery



SOAP

Distributed Component
Object Model (DCOM)

opalORB

XPath

ZEN

IDL

Remote Method
Invocation
(RMI)

Encina/9000

Rendezvous



ORBlite

Remote Procedure Call
(RPC)



BEA WebLogic®

ORACLE®



Extensible Markup Language (XML)

Distributed Sys **WebSphere.** software

